

UNIVERSIDAD DE GUADALAJARA



CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

Arquitectura de Computadoras

Nombre: Alonso Jaquez Sara Iriel

Maestro: Jorge Ernesto López Arce Delgado

Actividad 02: Compuertas Lógicas

Sección: D03

I. Introducción

En la materia de Arquitectura de Computadoras es fundamental comprender cómo funcionan los sistemas digitales desde su nivel más básico. Uno de los componentes principales de estos sistemas son las compuertas lógicas, ya que a partir de ellas se construyen circuitos más complejos como sumadores, registros y procesadores.

En esta práctica se llevó a cabo el diseño e implementación de distintas compuertas lógicas básicas utilizando el lenguaje de descripción de hardware Verilog, así como su simulación en el software ModelSim. El trabajo consistió en crear un módulo que recibe señales de entrada y genera las salidas correspondientes a cada compuerta lógica, permitiendo observar su comportamiento de acuerdo con las combinaciones posibles de entradas.

Mediante esta actividad se buscó aplicar los conocimientos teóricos adquiridos previamente, relacionándolos con una implementación práctica en un entorno de simulación. De esta manera, la práctica facilita la comprensión del funcionamiento real del hardware digital y la forma en que las compuertas lógicas pueden describirse y analizarse mediante un lenguaje de programación orientado al diseño de circuitos.

Implementación de compuertas lógicas.

Una FPGA es un dispositivo electrónico reconfigurable que permite implementar circuitos digitales directamente en hardware. Está formado por bloques lógicos programables, interconexiones configurables y elementos de entrada/salida. A diferencia de otros, este implementa físicamente el circuito lógico, permitiendo paralelismo real y alta velocidad.

El lenguaje de descripción de hardware VERILOG se utiliza para describir el comportamiento y la estructura de los circuitos digitales que se desean implementar dentro de la FPGA. Con VERILOG se especifican compuertas lógicas, módulos, registros y conexiones. Posteriormente, los herramientas de síntesis traducen el código a una red de compuertas y bloques lógicos que la FPGA puede configurar internamente.

Tablas de verdad de las compuertas lógicas.

Compuerta AND

A	B	AND
0	0	0
0	1	0
1	0	0
1	1	1

Compuerta OR

A	B	OR
0	0	0
0	1	1
1	0	1
1	1	1

Compuerta XNOR

A	B	XNOR
0	0	1
0	1	0
1	0	0
1	1	1

Compuerta NOT

A	NOT
0	1
1	0

Compuerta NAND

A	B	NAND
0	0	1
0	1	1
1	0	1
1	1	0

Compuerta NOR

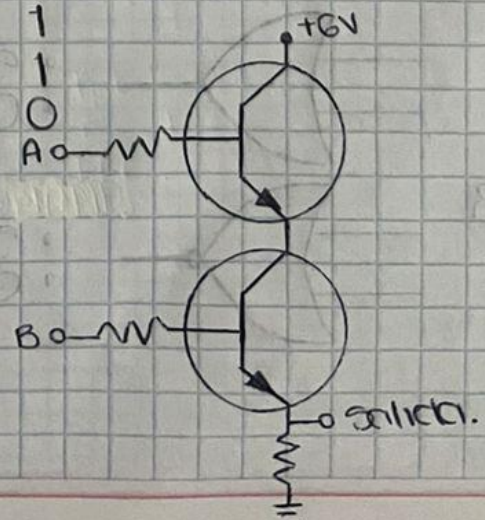
A	B	NOR
0	0	1
0	1	0
1	0	0
1	1	0

Compuerta XOR

A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Aireado de transistores.

Compuerta AND: En CMOS, se constituye un NAND y luego se añade un inversor (NOT). Se colocan en serie.



Alonso Torrez

Compuerta OR: Los transistores se colocan en paralelo

Compuerta NOT: Entrelazada (1), el transistor NMOS se conecta y conecta la salida a tierra (0). Cuando es lógico (0) el PMOS se conecta y conecta la salida a voltaje (1).

Compuerta NOR: Transistores NMOS están en paralelo hacia tierra y los PMOS en serie hacia el voltaje.

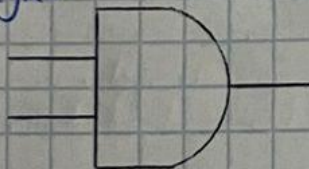
Compuerta NAND: Los transistores NMOS están en serie hacia tierra y los PMOS en paralelo hacia el voltaje.

Compuerta XOR: Combina varios compuertas (NAND, AND, OR) o una configuración específica de transistores de paso.

Compuerta XNOR: Varía según la tecnología usada.

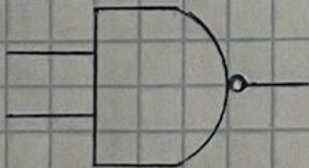
Simbología

AND



- Símbolo lógico: $\wedge$
- Operador en Verilog: $\&$

NAND



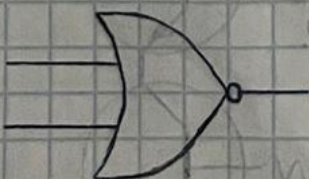
- Símbolo lógico: ∇
- Operador en Verilog: $\sim\&$

OR

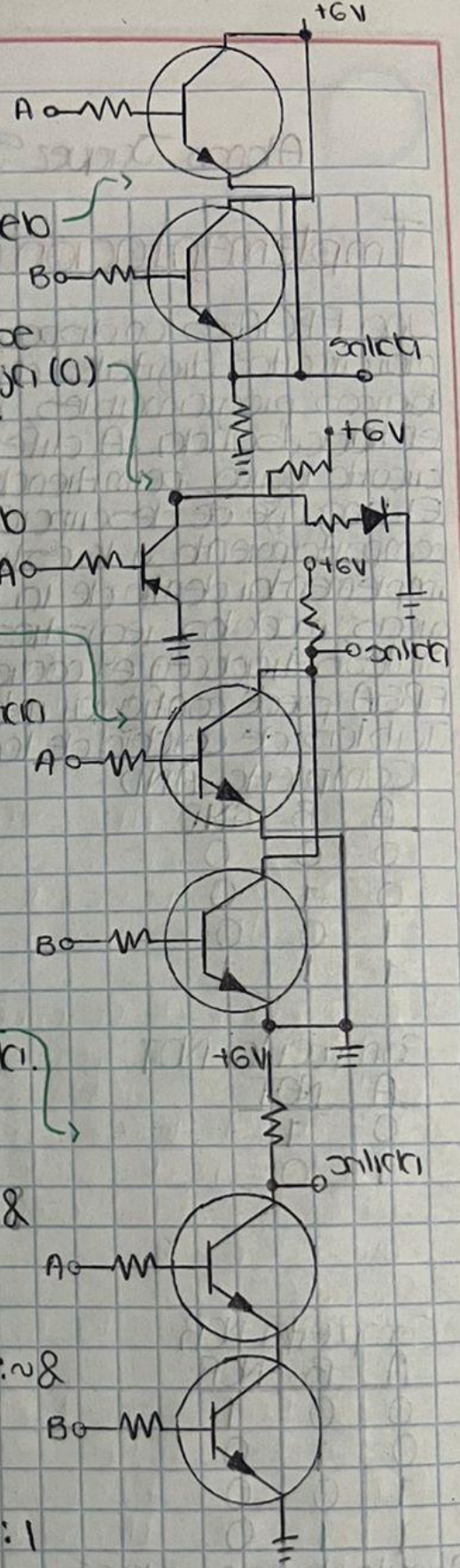


- Símbolo lógico: $\vee$
- Operador en Verilog: $\mid$

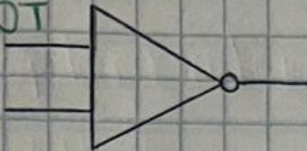
NOR



- Símbolo lógico: ∇
- Operador en Verilog: $\sim\mid$



NOT



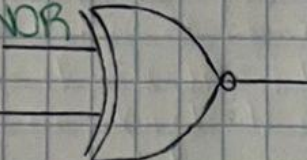
- Símbolo lógico: $\neg$
- Operador en Verilog: $\sim$

XOR



- Símbolo lógico: $\oplus$
- Operador en Verilog: $\wedge$

XNOR



- Símbolo lógico: $\odot$
- Operador en Verilog: $\sim \wedge$ o $\wedge \sim$

II. Objetivos

El objetivo de esta práctica es describir de manera detallada cómo se realizó la implementación de cada uno de los módulos desarrollados en el lenguaje de descripción de hardware Verilog, así como la lógica utilizada para representar el funcionamiento de las compuertas lógicas básicas. Se explica la estructura del módulo creado, la definición de entradas y salidas y la forma en que se implementó la lógica combinacional para cada compuerta.

De igual manera, se detalla el proceso seguido para desarrollar la práctica en el software ModelSim, explicando los elementos utilizados y la razón de su elección. Aunque no se emplean algoritmos complejos, se describe la lógica aplicada para obtener las salidas correctas a partir de las entradas, con el fin de demostrar que la actividad fue realizada y comprendida.

III. Desarrollo

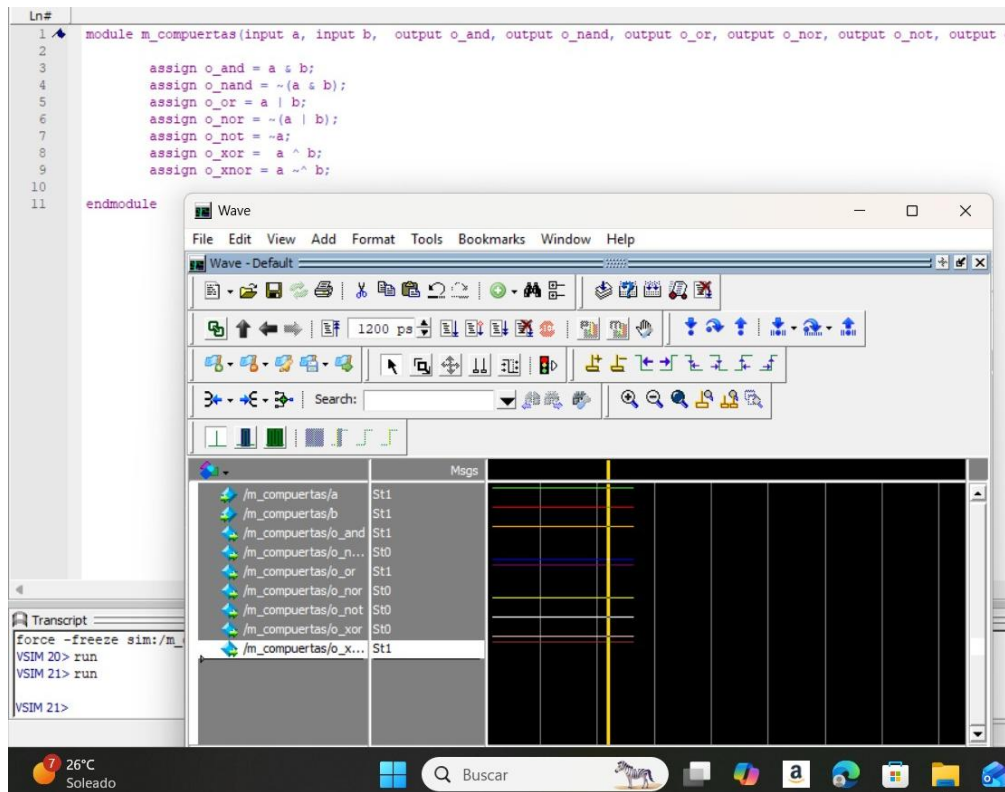
Para la realización de esta práctica se utilizó el software ModelSim, el cual permite escribir, compilar y simular código en Verilog. Se creó un módulo llamado `m_compuertas`, el cual recibe dos entradas lógicas (a y b) y genera varias salidas correspondientes a las compuertas lógicas implementadas.

El código fue desarrollado utilizando asignaciones continuas mediante la instrucción `assign`, ya que este tipo de descripción es adecuado para representar circuitos combinacionales, donde las salidas dependen únicamente de las entradas en un instante dado.

Cada compuerta lógica se implementó utilizando los operadores lógicos propios del lenguaje Verilog:

1. AND (&): Realiza la operación lógica Y entre las entradas a y b.
2. NAND (~& o negación del AND): Se obtiene negando el resultado de la compuerta AND.
3. OR (|): Realiza la operación lógica OR entre las entradas.
4. NOR (~|o negación del OR): Se obtiene negando la salida de la compuerta OR.
5. NOT (~): Niega el valor de la entrada a.
6. XOR (^): Produce un valor verdadero cuando las entradas son diferentes.
7. XNOR (~^): Produce un valor verdadero cuando las entradas son iguales.

El módulo fue probado en simulación para verificar que cada salida respondiera correctamente de acuerdo con las tablas de verdad correspondientes a cada compuerta lógica, confirmando así el correcto funcionamiento del diseño.



IV. Conclusión

La práctica permitió comprender de manera clara la relación entre la teoría de lógica digital y su implementación mediante un lenguaje de descripción de hardware como Verilog. A través del diseño del módulo de compuertas lógicas, se reforzó el uso de operadores lógicos y la estructura básica de un módulo combinacional.

El uso de ModelSim facilitó la simulación y validación del comportamiento del circuito, permitiendo observar cómo cambian las salidas en función de las entradas.

Finalmente, se puede concluir que el uso de Verilog y herramientas de simulación es esencial para el desarrollo y análisis de circuitos digitales, y que esta práctica contribuye significativamente al entendimiento del funcionamiento interno del hardware digital.

V. Referencias

Logic Gates Verilog Code - Circuit fever. (n.d.). <https://circuitfever.com/logic-gates-verilog-code>

Instructables. (2017, October 17). *Logic gates with NPN transistors.* Instructables. <https://www.instructables.com/Logic-Gates-with-NPN-transistors/>

Wtdtantakatan. (2016, September 21). *Transistores en compuertas lógicas.* Tantakatanblog. <https://tantakatanblog.wordpress.com/2016/09/18/transistores/>